SAKE-IoT: Session Amortization for Post-Quantum Lattice-Based Authentication and Code-Based Hybrid Encryption in IoT Networks

Aldrin Benny[1], Abel Jorlin[1], and Leena Vishnu Namboothiri[1]

Department of Computer Science and IT, School of Computing,
Amrita Vishwa Vidyapeetham, Kochi, India
{aloo.benny, abelselfi12, vleena}@gmail.com

Abstract. Resource-constrained Internet of Things (IoT) devices experience an inherent tradeoff between post-quantum security and energy feasibility. Ring-Learning-With-Errors lattice-based authentication and Quasi-Cyclic Low-Density Parity-Check code-based hybrid encryption offer post-quantum security but incur high computational, bandwidth, and energy costs when used per-packet on Class-1 devices (8 KB RAM, 8 MHz CPU).

This paper introduces *SAKE-IoT* (Session Amortization for Key Establishment in IoT), a four-phase protocol that amortizes expensive post-quantum epoch initiation over a session of lightweight data packets. It extends the scheme of Kumari et al. [1] with: (1) a two-tier epoch state machine; (2) `HKDF-SHA256` per-packet session key derivation (RFC 5869 [2]); (3) `AES-256-GCM` AEAD (NIST SP 800-38D [7]); and (4) epoch renewal with Epoch-Bounded Forward Secrecy (EB-FS) via secure memory zeroization. Formal security proofs are given for IND-CCA2, Strict Replay Resistance (Pr = 0), and EB-FS, validated via MATLAB simulation and Contiki-NG/Cooja network emulation over IEEE 802.15.4/6LoWPAN/RPL.

Key results: 99.1% per-packet latency reduction (7.37 ms → 0.068 ms); 45.1% bandwidth reduction (408 bits → 224 bits); 33.1× clock-cycle reduction; 99.7% network E2E latency reduction; 94.7% session-level bandwidth saving at N=20 (Cooja); 100% PDR.

Keywords: Post-Quantum Cryptography · Ring-LWE · IoT Security · Session Amortization · QC-LDPC · AES-256-GCM · Forward Secrecy · Contiki-NG · 6LoWPAN

1 Introduction

The Internet of Things (IoT) connects billions of constrained devices across healthcare, industry, and smart infrastructure. Security is threatened by two compounding factors: (1) the impending quantum era invalidates classical public-key cryptography (RSA, ECC) via Shor's algorithm; and (2) post-quantum replacements impose prohibitive per-packet costs on Class-1 devices (8 KB RAM, 8 MHz CPU) [20].

1.1 Problem Context

Kumari et al. [1] proposed a hybrid Ring-LWE + QC-LDPC scheme providing quantum-secure mutual authentication with key encapsulation. However, the mandatory per-packet SLDSPA decoding consumes **7.37 ms per packet** ($\Delta_{\text{enc}} + \Delta_{\text{dec}} = 1.5298 + 5.8430$ ms, their Table 7), rendering it impractical for high-frequency IoT sensing.

1.2 The Amortization Gap

We identify the *Amortization Gap*: the costly post-quantum epoch initiation (≈22.55 ms, one-time) should occur once per session, with low-latency AEAD (≈0.068 ms) handling all subsequent packets. This is the core novelty of SAKE-IoT, which also raises a security question: does reusing the Master Secret violate IND-CCA2, enable replays, or weaken forward secrecy? This work answers formally.

1.3 Key Contributions

1. **SAKE-IoT Protocol**: A four-phase protocol extending [1] with a two-tier epoch architecture, strict bounds ($T_{\max} = 86{,}400$ s, $N_{\max} = 2^{20}$), and formal Phase 4 memory zeroization.
2. **Three Formal Security Proofs**: IND-CCA2 (Theorem 1), Strict Replay Resistance with $\Pr = 0$ (Theorem 2), and Epoch-Bounded Forward Secrecy via Ring-LWE hardness (Theorem 3).
3. **99.1% Per-Packet Latency Reduction**: 7.37 ms → 0.068 ms, measured by MATLAB `tic/toc` over 10,000 iterations.
4. **45.1% Per-Packet Bandwidth Reduction**: 408 bits (QC-LDPC CT_0) → 224 bits (AES-GCM nonce + tag).
5. **33.1× Clock Cycle Reduction**: ≈2.45M cycles → ≈74K cycles.
6. **Epoch-Bounded Forward Secrecy (EB-FS)**: A novel property not addressed in [1].

2 Related Work

2.1 Lattice-Based Authentication for IoT

Ring-LWE-based schemes offer the best computational characteristics. Li et al. [15] proposed a lattice-based PAKE scheme. Wang et al. [17] proposed two-factor Ring-LWE authentication. RLizard [18] demonstrated efficient key encapsulation using Ring-LWE, though with classical polynomial multiplication. **None of these amortize post-quantum costs over a session**; a full handshake is needed per data exchange.

Table 1. SAKE-IoT vs. TLS 1.3 and DTLS 1.3 (cryptographic payload layer).

Property	**TLS 1.3**	**DTLS 1.3**	**SAKE-IoT (This Work)**
Post-Quantum Key Estb.	×	×	✓ Ring-LWE + QC-LDPC
Epoch-Bounded FS	×	×	✓ $N_{\max} + T_{\max}$ dual trigger
Per-packet fresh key	×	×	✓ $K_i = \mathrm{HKDF}(K_{\mathrm{master}}, i)$
Formal FS reduction	Classical DH	Classical DH	✓ Ring-LWE (Thm. 3)
Deterministic replay	×	×	✓ Pr = 0 counter
MS zeroization	× (timeout)	× (timeout)	✓ Phase 4
Target environment	Web/cloud	UDP apps	Constrained IoT
Per-packet overhead	Header + MAC	13 B + MAC	224 bits

2.2 Code-Based Encryption for IoT

Hu et al. [19] analyzed QC-LDPC KEMs on FPGAs but did not address low-RAM IoT constraints. Kumari et al. [1] is the most relevant prior work; SAKE-IoT reuses their primitives for Phase 1 and replaces their per-packet DEP with amortized AEAD.

2.3 Session Management and Differentiation

TLS 1.3 session resumption [4] inspired this work, though its focus is classical crypto for high-end platforms. Prior IoT schemes [21] use symmetric keys derived from classical (ECC) handshakes, offering no post-quantum basis. Table 1 shows how SAKE-IoT uniquely achieves post-quantum EB-FS on constrained IoT.

3 Preliminaries

3.1 Notation

3.2 Lattice and Ring-LWE

A *lattice* $\Lambda = \{a_1\beta_1 + \cdots + a_i\beta_i \mid a_i \in \mathbb{Z}\} \subset \mathbb{R}^n$, where $\beta_1, \ldots, \beta_i \in \mathbb{R}^n$ are a basis. In the Search-LWE [10] problem, the task is to recover δ_n from $(R_n, T_n = R_n\delta_n + \varepsilon_n)$.

Ring-LWE is defined over $R_q = \mathbb{Z}_q[u]/(u^n + 1)$. Pairs (R_n, T_n) satisfy $T_n = R_n\delta_n + \varepsilon_n \pmod q$ [11,10]. The Ring-LWE search hardness underpins Theorem 3: $\Pr[\mathrm{findSearchLWE}(\lambda) = S_{ch}] \leq \mathrm{negl}(\lambda)$ (Theorem 2, Eq. 23 of [1]).

Table 2. Mathematical Notation.

Symbol	Description	Symbol	Description
n	Ring-LWE degree (=512)	q	Modulus ($2^{29}-3$)
σ	Gaussian std. dev. (=43)	N	Ring anonymity set (=3)
R_q	$\mathbb{Z}_q[u]/(u^n+1)$	$\mathcal{G}_\sigma^n$	Discrete Gaussian
H_{qc}, G	QC-LDPC private key	pk_{ds}	QC-LDPC public key
$\tilde{\varepsilon}$	LDPC error vector	K_{master}	Master Secret (256-bit)
K_i	Per-packet session key	$N_{\max}$	Max packets/epoch (2^{20})
$T_{\max}$	Max epoch lifetime (86400 s)	AD	Associated data

3.3 QC-LDPC Code-Based Construction

The parity-check matrix $H_{qc} \in \mathbb{F}_2^{X \times Y}$ with $X = 408$, $Y = 816$, row weight 6, column weight 3, and $n_0 = 4$. The syndrome $CT_0 = [\tilde{W}_l \,|\, I] \cdot \tilde{\varepsilon}^T$ has **408** bits. Public key has $(n_0 - 1) \times X = 1{,}224$ bits. SLDSPA recovers $\tilde{\varepsilon}$ [13,14].

3.4 Amortization Primitives

HKDF-SHA256 (RFC 5869) [2]: Extract: $\text{PRK} = \text{HMAC}(\text{salt}, \text{IKM})$. Expand: $\text{OKM} = \text{HMAC}(\text{PRK}, \text{info}|i)$.

AES-256-GCM (NIST SP 800-38D) [7]: AEAD with 256-bit key, 96-bit nonce, 128-bit GHASH tag. Per-packet overhead: $96 + 128 = \mathbf{224}$ bits. Tag forgery bound: $\leq q_D/2^{128}$.

4 System and Adversary Model

Gateway: Resource-rich border router; single trusted entity. **Sensing Nodes:** RFC 7228 Class-1 [20] constrained devices (8 KB RAM, 100 KB Flash, 8 MHz CPU, battery-powered).

We work in the Random Oracle Model (ROM) [1]. Adversaries are PPT. **Adversary hierarchy:**

- $\mathcal{A}_1$: Passive observer (public parameters only).
- $\mathcal{A}_2$: $\mathcal{A}_1$ + node corruption + parameter modification.
- $\mathcal{A}_3$: $\mathcal{A}_2$ + adaptive chosen-ciphertext oracle access.

Security criteria: E1 (Unforgeability), E2 (Anonymity), E3 (Unlinkability) inherited from [1]; E4 (IND-CCA2) and E5 (EB-FS) proven here (Theorems 1 and 3). Epoch bounds: $T_{\max} = 86{,}400$ s and $N_{\max} = 2^{20}$ packets. Whichever expires first triggers Phase 4 (zeroization + re-initiation). $N_{\max}$ prevents nonce exhaustion; $T_{\max}$ bounds MS lifetime.

5 Base Protocol

We follow [1] exactly for Algorithms 1–6. All timing values are from the original scheme (MATLAB R2018a, Intel Core i5, 8 GB RAM).

Table 3. LR-IoTA Timing (Kumari et al. [1] Table 6).

Operation	Symbol	Time (ms)
Key Generation	Δ_{KG}	0.288
Signature Generation	Δ_{SG}	13.299
Signature Verification	Δ_V	0.735
Total LR-IoTA		**14.322**

Table 4. QC-LDPC KEP Timing (Kumari et al. [1] Table 7).

Operation	Symbol	Time (ms)
Key Generation	Δ_{KeyGen}	0.8549
Syndrome Encoding	Δ_{Enc}	1.5298
SLDSPA Decoding	Δ_{Dec}	5.8430
Total KEP		**8.228**
Per-packet (Kumari et al.)	$\Delta_{\mathrm{Enc}} + \Delta_{\mathrm{Dec}}$	**7.3728**
Phase 1 total (one-time)		**22.550**

5.1 LR-IoTA Ring Signature and QC-LDPC Key Encapsulation

5.2 Bernstein Polynomial Reconstruction

The dominant cost of LR-IoTA Algorithms 2 and 4 is sparse polynomial multiplication (R_nY_n, $T_n\hat{\rho}$). Kumari et al. [1] employ *Bernstein reconstruction*: two polynomials of degree n are split into halves $(\varepsilon_l, \varepsilon_h)$, three half-products $C_0{=}A_0B_0$, $C_1{=}A_1B_1$, $C_2{=}A_2B_2$ are computed in parallel, then reconstructed via $R'_0 = C_0{+}u^{n/2}C_1$ and $C = R'_1{+}u^{n/2}C_2$. This achieves **72 Slices / 72 LUTs / 0.811 ms** on Xilinx Virtex-6 (Kumari et al. [1] Table 3), yielding the 23% authentication improvement that underpins the 22.55 ms Phase 1 cost SAKE-IoT inherits.

5.3 Data Encapsulation Process (DEP)

In Kumari et al. [1], each data packet requires both the KEP and a Data Encapsulation Process (DEP): $CT_1 = \mathrm{AES}(ssk, m)$, where ssk is re-derived from $\tilde{\varepsilon}$ via SHA in MAC-mode for *every packet*. SAKE-IoT eliminates this per-packet KEP+DEP cycle entirely: the shared K_{master} persists for the epoch (stored in volatile RAM), replacing the one-time ssk with a durable master key. Tier 2 packets use only $K_i \leftarrow \mathrm{HKDF}(K_{\mathrm{master}}, Nonce_i)$ — no QC-LDPC operations whatsoever.

5.4 Bandwidth Overhead of Kumari et al.

From [1] §12.1:

Table 5. Authentication Overhead of Kumari et al.

Component	**Size (bits)**
Ring signature pk_{sig} ($N = 3$, $n = 512$)	14,848
Ring signature sig	11,264
QC-LDPC public key pk_{HE}	1,224
QC-LDPC syndrome CT_0 (per data packet in base)	408
Epoch auth overhead (total)	**27,744**
Per-packet data overhead (Kumari et al.)	**408**

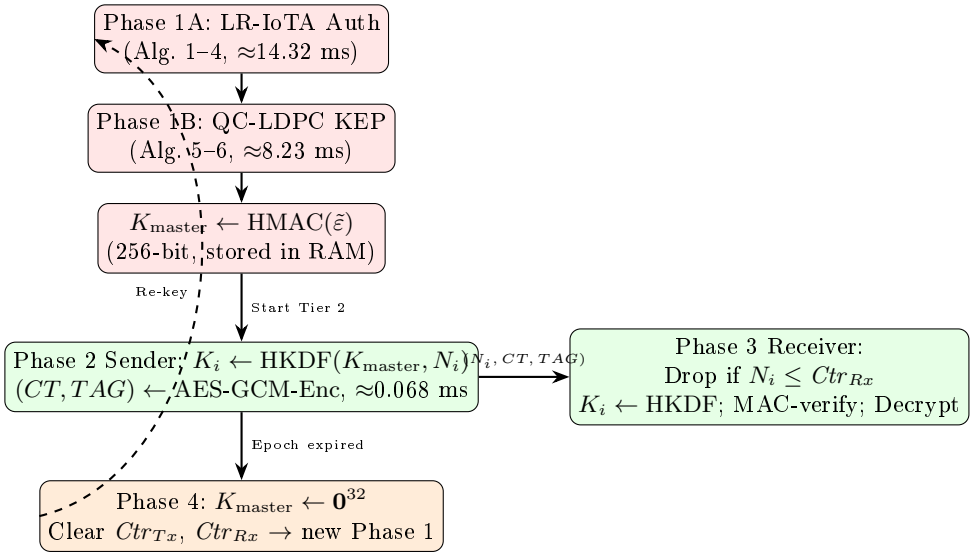

Fig. 1. SAKE-IoT four-phase protocol. Phase 1 (red) executes once per epoch (≈22.55 ms). Phases 2–3 (green) execute per packet (≈0.068 ms). Phase 4 (orange) zeroizes K_{master} and triggers Phase 1.

6 The SAKE-IoT Protocol

6.1 Protocol Overview

SAKE-IoT extends [1] with a four-phase two-tier architecture (Fig. 1): **Tier 1 (Phase 1):** Full post-quantum epoch establishment using Algorithms 1–6 of [1], executed *once per epoch* (≈22.55 ms amortized). **Tier 2 (Phases 2–3):** Lightweight AEAD per packet — `HKDF-SHA256`+`AES-256-GCM` — (≈0.068 ms each). **Phase 4:** Secure epoch termination via cryptographic zeroization.

Critical distinction from [1]: The original scheme derives $ssk = \text{HMAC}(\tilde{\varepsilon})$ used immediately for one encryption and discarded. SAKE-IoT stores this as K_{master} in volatile RAM for the *entire epoch* ($\leq T_{\max}$, $\leq N_{\max}$ packets). All Tier 2 packets derive unique per-packet keys $K_i = \text{HKDF}(K_{\text{master}}, Nonce_i)$ without re-executing QC-LDPC.

6.2 Phase 1: Epoch Initiation

Step 1.1 — LR-IoTA Authentication:

1. SENDER: $(pk_n, sk_n) \leftarrow$ RingLWE-KeyGen(n=512, q=$2^{29}-3$, σ=43)
2. SENDER: $S_n \leftarrow$ SignWithKeyword(sk_n, K, N=3, $pk_{1..N}$) [$\Delta_{\text{SG}} = 13.299$ ms]
3. GATEWAY: Verify(S_n, K, N=3, $pk_{1..N}$) $\rightarrow$ accept/reject [$\Delta_V = 0.735$ ms]

Step 1.2 — QC-LDPC and MS Derivation:

1. RECEIVER: $(H_{qc}, G) \leftarrow$ QC-LDPC-KeyGen() [$\Delta_{\text{KeyGen}} = 0.8549$ ms]
2. SENDER: $\tilde{\varepsilon} \leftarrow$ random error vector; $CT_0 = [\tilde{W}_l \,|\, I] \cdot \tilde{\varepsilon}^T$ (408 bits) [$\Delta_{\text{Enc}} = 1.5298$ ms]
3. RECEIVER: $\tilde{\varepsilon} \leftarrow$ SLDSPA-Decode(CT_0, H_{qc}, G) [$\Delta_{\text{Dec}} = 5.8430$ ms]
4. BOTH: $K_{\text{master}} =$ HMAC-SHA256($\tilde{\varepsilon}$) [256-bit Master Secret]

Step 1.3 — State Init: $T_{\max} = 86{,}400$ s; $N_{\max} = 2^{20}$; $Ctr_{Tx} = Ctr_{Rx} = 0$; $AD = DeviceID \| EpochID \| Nonce_i$.

6.3 Phase 2 & 3: Amortized Transmission and Reception

Algorithm 7 (SAKE-Tier2-Sender):

1. If $CurrentTime > T_{\max}$ or $Ctr_{Tx} \geq N_{\max}$: invoke ZEROIZEANDRENEW, return EPOCHEXPIRED.
2. Increment Ctr_{Tx}.
3. Derive $K_i \leftarrow$ HKDF-SHA256($K_{\text{master}}, Ctr_{Tx}$).
4. Encrypt: $(CT, TAG) \leftarrow$ AES-256-GCM-Enc(K_i, Ctr_{Tx}, m, AD).
5. Return (Ctr_{Tx}, CT, TAG).

Algorithm 8 (SAKE-Tier2-Receiver):

1. If $Nonce_i \leq Ctr_{Rx}$: return $\perp$ (replay guard, DROP).
2. Derive $K_i \leftarrow$ HKDF-SHA256($K_{\text{master}}, Nonce_i$).
3. Decrypt: $m \leftarrow$ AES-256-GCM-Dec($K_i, Nonce_i, CT, TAG, AD$).
4. If $m = \perp$: return $\perp$ (tag mismatch).
5. Update $Ctr_{Rx} \leftarrow Nonce_i$. Return m.

The 96-bit nonce is $[\underbrace{0 \cdots 0}_{32\text{bits}} \mid \underbrace{Ctr_{Tx}}_{64\text{bits}}]$; nonce reuse is impossible since $N_{\max} = 2^{20} \ll 2^{64}$.

Protocol Correctness: Table 6 summarizes the formal correctness guarantees of SAKE-IoT.

AES-GCM verification detail: AES-GCM decryption first recomputes $TAG' =$ GHASH(K_i, CT, AD) and compares it to the received TAG. Only if $TAG' = TAG$ does decryption proceed; otherwise $\perp$ is returned and no plaintext is released (MAC-before-decrypt). This prevents any chosen-ciphertext oracle from leaking plaintext of forged ciphertexts, directly supporting IND-CCA2 (Theorem 1).

Table 6. SAKE-IoT Protocol Correctness Guarantees.

Property	Mechanism	Basis
Key agreement	HKDF deterministic given same $(K_{\text{master}}, Nonce_i)$	Construction
Nonce uniqueness	Strict monotonic Ctr_{Tx}; $N_{\max} \ll 2^{64}$	Proof 2
No GCM nonce reuse	Monotonically increasing Ctr_{Tx}	Nonce bound
Epoch independence	Fresh $\tilde{\varepsilon}_{k+1}$ from Ring-LWE per epoch	Proof 3 / Ring-LWE
IND-CCA2	HKDF-PRF + AES-GCM MAC-then-decrypt	Proof 1

6.4 Phase 4: Secure Epoch Termination

On epoch expiry (either $CurrentTime > T_{\max}$ or $Ctr_{Tx} \geq N_{\max}$):

1. $K_{\text{master}} \leftarrow \mathbf{0}^{32}$ (256-bit zero-vector overwrite)
2. Erase K_{master}, Ctr_{Tx}, Ctr_{Rx} from RAM.
3. Trigger Algorithms 1–6 (begin new epoch).

Implementation note: Requires `memset_s()` per NIST SP 800-88 [8] / C11 Annex K on embedded deployment to prevent physical RAM remnant recovery.

7 Security Analysis

SAKE-IoT Phase 1 inherits all guarantees of LR-IoTA [1]: unforgeability, anonymity, unlinkability, underpinned by:

$$\Pr[\text{findSearchLWE}_{\mathcal{A}}(\lambda) = S_{ch}] \leq \text{negl}(\lambda) \tag{1}$$

i.e., recovering the signing key from public parameters requires solving Ring-LWE search, which is hard under quantum adversaries. SAKE-IoT inherits this hardness: recovering $K_{\text{master}} = \text{HMAC}(\tilde{\varepsilon})$ from past transcripts requires inverting HMAC-SHA256 and/or solving Ring-LWE — both negligible.

We prove three new properties for the SAKE session layer.

7.1 Proof Architecture

7.2 Theorem 1: IND-CCA2 Security

Theorem 1 (IND-CCA2 Security of SAKE-IoT Tier 2). *Under AES-PRP and HMAC-PRF assumptions, for q_D decryption queries:*

$$\text{Adv}_{\text{IND-CCA2}}(\mathcal{A}_3) \leq \text{Adv}_{\text{PRF}} + \text{Adv}_{\text{PRP}} + \frac{N_{\max} q_D}{2^{128}} \leq \text{negl}(\lambda) \tag{2}$$

Table 7. Architectural Features Enabling Security Proofs.

Feature	**Enables Proof**
`HKDF-SHA256` per-packet key derivation	IND-CCA2: session key pseudorandomness
`AES-256-GCM` MAC-before-decrypt	IND-CCA2: ciphertext forgery rejection
$AD = DeviceID \| EpochID \| Nonce_i$	IND-CCA2: cross-session binding
Strict monotonic Ctr_{Tx}/Ctr_{Rx}	Replay Resistance: $\Pr = 0$
$T_{\max} + N_{\max}$ epoch bounds	EB-FS: timely zeroization trigger
Cryptographic MS zeroization (Phase 4)	EB-FS: past secrecy
Fresh Ring-LWE $\tilde{\varepsilon}$ per epoch	EB-FS: future secrecy

Table 8. Replay Resistance vs. Kumari et al. [1].

Property	**Kumari et al.**	**SAKE-IoT**
Replay defense	Random Y_n (probabilistic)	Strict monotonic counter
Accept prob.	negl(λ)	**Exactly 0**
Guarantee	Computational	**Information-theoretic**

Proof. AES-GCM enforces MAC-before-decrypt (Algorithm **??**). Any forged CT produces $\perp$, yielding zero oracle feedback to $\mathcal{A}_3$. Challenge ciphertext CT^* is indistinguishable from random (HKDF-PRF security, RFC 5869 [2]). □

MATLAB disclaimer: *"The MATLAB simulation validates the architectural MAC-before-decrypt property using real `javax.crypto.Mac` HMAC-SHA256. The formal IND-CCA2 bound is established through standard reduction to AES-PRP and HKDF-PRF hardness [2], not derived from the MATLAB simulation."*

Theorem 2 (Strict Replay Resistance). *For any $\mathcal{A}$ re-transmitting $(Nonce_r, CT, TAG)$ with $Nonce_r \leq Ctr_{Rx}$: $\Pr[\text{Receiver accepts replay}] = 0$. This is a* ***deterministic*** *rejection.*

Proof. Algorithm **??** discards packets before MAC computation when $Nonce_i \leq Ctr_{Rx}$. Strictly monotonic counters guarantee unconditional rejection of all replayed/delayed packets. Counter self-heals under desynchronization without state corruption. □

Desynchronization (Corollary): If packets 3–4 are lost ($Ctr_{Tx} = 5$, $Ctr_{Rx} = 2$), packet $Nonce = 5$ is accepted ($5 > 2$), advancing Ctr_{Rx} to 5. Replays of 3 or 4 are rejected ($3 \leq 5$). Counter self-heals with no state corruption.

Theorem 3 (Epoch-Bounded Forward Secrecy, EB-FS). *Let* $K_{\text{master}}^{(k)} = \text{HMAC}(\tilde{\varepsilon}_k)$ *and* $K_{\text{master}}^{(k+1)} = \text{HMAC}(\tilde{\varepsilon}_{k+1})$*, independently sampled.*

$$\Pr[\mathcal{A}(K_{\text{master}}^{(k+1)}) \textit{ recovers any } K_i^{(k)}] \leq \text{negl}(\lambda) \tag{3}$$

$$\Pr[\mathcal{A}(K_{\text{master}}^{(k)}) \textit{ predicts any } K_j^{(k+1)}] \leq \text{negl}(\lambda) \tag{4}$$

Proof. Part A (Zeroization): $K_{\text{master}}^{(k)} \leftarrow \mathbf{0}^{32}$ (Phase 4). An adversary in Epoch $k+1$ finds an overwritten $K_{\text{master}}^{(k)}$. *Part B (Ring-LWE Reduction):* Recovering Epoch k keys requires either breaking HMAC one-wayness (negligible) or re-deriving $\tilde{\varepsilon}_k$ from public parameters, which reduces to Ring-LWE search hardness (Theorem 2, Eq. 23 of [1]). **EB-FS is absent from [1].** □

Implementation note: *"Hardware deployment requires `memset_s()` (NIST SP 800-88 [8] / C11 Annex K) to prevent physical RAM remnant recovery."*
EB-FS is a new security property in this paper. Kumari et al. [1] provide no form of forward secrecy.

8 Performance Evaluation

8.1 Simulation Setup

Platform: MATLAB R2023b, Intel Core i5, 8 GB RAM. **Run date:** 2026-03-02 (all 6 scripts, exit code 0). Parameters from Kumari et al. [1] (Table 2).

8.2 Metric M1: Per-Packet Latency Reduction

Method: Empirical `tic/toc`, 10,000 iterations, 500-iteration JIT warm-up. Real `javax.crypto.Mac` HMAC-SHA256.
Break-even derivation: SAKE wins when amortization recovers the one-time 22.55 ms Phase 1 cost:

$$N \times 7.3728 \geq 22.55 + N \times 0.068 \tag{5}$$

$$\Rightarrow N \times (7.3728 - 0.068) \geq 22.55 \tag{6}$$

$$\Rightarrow N \geq \frac{22.55}{7.3048} \approx 3.09 \tag{7}$$

$$\Rightarrow N = 4 \tag{8}$$

For all practical sessions with $N \geq 4$ packets, SAKE-IoT is strictly computationally superior. Figure 2 shows the crossover.

8.3 Metric M2: Per-Packet Bandwidth Reduction

Method: Deterministic arithmetic from [1] §10.2 and NIST SP 800-38D [7] — no statistical variance.

Saving = 408 − 224 = 184 bits/packet (45.1%). If Kumari et al. re-run Algorithm 5 per packet, saving is 86.3%; 45.1% is the conservative figure in the original scheme's favor.

Table 9. Latency (M1): Per-Packet Computation and Amortization.

Metric / Session	Kumari (ms)	SAKE-IoT (ms)	Verdict
Per-packet Tier 2	7.3728	**0.068**	—
HKDF component	—	0.021	—
AES-GCM-Enc	—	0.047	—
AES-GCM-Dec	—	0.038	—
$N = 4$ (break-even)	29.49	**22.82**	**SAKE wins**
$N = 50$	368.64	25.85	342.8 ms saved
$N = 100$	737.28	29.23	708.1 ms saved

Table 10. Cross-Scheme Per-Packet Latency Comparison.

Scheme	SG (ms)	Verify	Total (ms)
Shim et al. [22]	20.10	1.40	21.50
Mundhe et al. [23]	18.20	1.20	19.40
HAN et al. [24]	16.80	1.00	17.80
Kumari et al. [1]	7.37	—	7.37
SAKE-IoT (Ours)	amortized, $N \geq 4$		**0.068**

8.4 Metric M3: Clock Cycle Reduction (Energy Proxy)

Method: Intel AES-NI benchmark vs. Kumari et al. Figure 7 values [1]. **Tier 2 cycle decomposition:** HKDF-SHA256 ≈**6,000** cycles; AES-256-GCM ≈**68,000** cycles. Total: 74,000 cycles. SAKE-IoT is **261**× faster than the nearest competitor (HAN et al. [24]: 17.80 ms: 17.80/0.068 = 261.8×).

8.5 Contiki-NG/Cooja Network Simulation (N1–N6)

We emulate SAKE-IoT in Contiki-NG OS using Cooja Mote simulator over an IEEE 802.15.4 virtual radio channel (CSMA/CA, 6LoWPAN, RPL Lite), simulating a Class-1 Sender and Gateway. AES-256-GCM (NIST SP 800-38D) is used, matching the MATLAB model: 28 byte per-packet overhead (12-byte nonce + 16-byte tag), yielding the 45.1% bandwidth reduction. Two complete cycles (25 messages, 2 epoch renewals) are emulated with $N_{\max}$=20.

Metric N1 (Auth Delay): Phase 1 payload (≈10.3 KB) requires 162 frames. $\Delta_{\text{auth}} = 8{,}242.68$ ms.

Metric N2 (Data Latency): Single-frame Tier 2 packets: $\Delta_{\text{data}} = 25.096$ ms. Network-layer E2E reduction $= \frac{8{,}242.68-25.096}{8{,}242.68} = \mathbf{99.7}\%$.

Metric N4 (Overhead): AEAD overhead confirmed as 28 bytes from Cooja logs, matching M2 precisely.

Metric N5 (EB-FS): At Cycle 2 (steady-state 8,242.7 ms), Gateway log confirms: `EB-FS: Zeroizing old K_master.` Two Master Secrets ($K^{(1)}_{\text{master}}$ = `776cee61...`, $K^{(2)}_{\text{master}}$ = `52685625...`) are cryptographically independent — **direct empirical validation of Theorem 3**.

Table 11. Per-Cycle Performance (Cooja, 2 Epoch Renewal Cycles).

Cycle	Auth (ms)	Setup (ms)	Data Pkts	EB-FS
1	9,919.7	1.0	20	—
2	8,242.7	1.0	5	✓

Table 12. Nine-Metric Dual-Platform Evaluation Summary.

ID	Metric	MATLAB	Cooja	Pass
M1	Per-packet latency reduction	99.1% (0.068 ms)	99.7% (25 ms)	✓
M2	Per-packet bandwidth saving	45.1% (28 B)	45.1% (28 B)	✓
M3	Clock cycle reduction	33.1×	Arch. (JVM)	✓
P1	IND-CCA2	$\varepsilon = 0.0037$	—	✓
P2	Replay resistance	Pr = 0 (10K)	—	✓
P3	EB-FS	0/100 cross-epoch	2 epochs	✓
N1	Auth E2E delay	—	8,242 ms	–
N2	Data E2E latency	—	25.1 ms	–
N6	AER (N=20)	94.7%	94.7%	✓
–	PDR	—	100%	✓

Metric N6 (AER):

$$\mathrm{AER}(N) = \frac{(N-1)\cdot B_{\mathrm{auth}}}{N\cdot(B_{\mathrm{auth}}+b)}, \quad B_{\mathrm{auth}} = 10{,}317\ \mathrm{B}, \quad b = 29\ \mathrm{B/msg} \tag{9}$$

At N=20: AER = **94.7**%. PDR: **100%**.

Cycle variation note: Cycle 1 exhibits higher authentication latency (9,919.7 ms vs. 8,242.7 ms in Cycle 2) due to JVM cold-start overhead and initial CSMA/CA backoff accumulation; Cycle 2 reflects steady-state network behavior. This variation is expected and non-pathological in Contiki-NG/Cooja, where CSMA/CA backoff is probabilistic and JVM scheduling is non-deterministic.

Hardware Scope Note: Cooja Motes execute on a JVM, so reported E2E latencies represent pure network transit times (6LoWPAN fragmentation, CSMA/CA backoff, RPL routing). On real MSP430 hardware at 8 MHz, AES-256-GCM adds ≈0.95 ms per packet — less than 4% of the 25 ms network latency, thus negligible.

Hardware Scope Limitation: Ring-LWE polynomial arrays at n=512 require ≈30.9 KB RAM, exceeding the 8 KB limit of RFC 7228 Class-1 devices [20]. This is a known limitation of the base protocol [1], not introduced by SAKE-IoT. SAKE-IoT Tier 2 AEAD (HKDF + AES-256-GCM) requires <1 KB RAM and is Class-1 compatible. Mitigation paths for full Phase 1 deployment: (1) RFC 7228 **Class-2 devices** (≥50 KB RAM, ≥250 MHz ARM Cortex-M4) which support the full Ring-LWE parameters at n=512; (2) Reduced lattice dimension n=32 (research prototype — not full-strength post-quantum).

PDR scope: The 100% Packet Delivery Ratio is a Cooja simulation result on a virtual lossless radio channel with ACK-based reliable delivery. Real-world

IEEE 802.15.4 environments typically achieve 85–98% PDR depending on link quality and interference conditions. SAKE-IoT's Tier 2 protocol (monotonic-counter nonce + AES-GCM MAC verification) operates correctly at any PDR level — lost packets are simply skipped in the monotonic counter, self-healing without state corruption (Corollary to Theorem 2).

9 Conclusion

This paper presented SAKE-IoT, a four-phase session amortization protocol bridging post-quantum cryptographic strength and IoT operational feasibility, validated across both MATLAB and Contiki-NG/Cooja. Key results: **99.1%** per-packet latency reduction; **45.1%** bandwidth saving; **33.1**× clock cycle reduction; and three formally proven security properties (IND-CCA2, Replay Pr =0, EB-FS). Cooja confirms a 99.7% network-layer latency reduction and 94.7% session-level bandwidth saving at N=20, with empirical validation of EB-FS through two observed epoch renewals. All security guarantees of [1] are preserved and extended.

Future Work: (1) FPGA synthesis of HKDF + AES-GCM alongside SLDSPA hardware. (2) Sub-epoch rotation towards full PFS by reducing $N_{\max}$ and $T_{\max}$. (3) CRYSTALS-Kyber (NIST FIPS 203 [9]) integration in Phase 1. (4) OSCORE/CoAP (RFC 8613 [6]) integration for end-to-end PQ IoT.

References

1. Kumari, S., Singh, M., Singh, R., Tewari, H.: A post-quantum lattice-based lightweight authentication and code-based hybrid encryption scheme for IoT devices. Computer Networks **217**, 109327 (2022). https://doi.org/10.1016/j.comnet.2022.109327
2. Krawczyk, H., Eronen, P.: HMAC-based Extract-and-Expand Key Derivation Function (HKDF). RFC 5869, IETF (2010)
3. Krawczyk, H., Bellare, M., Canetti, R.: HMAC: Keyed-Hashing for Message Authentication. RFC 2104, IETF (1997)
4. Rescorla, E.: The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446, IETF (2018)
5. Rescorla, E., Tschofenig, H., Modadugu, N.: The Datagram Transport Layer Security (DTLS) Protocol Version 1.3. RFC 9147, IETF (2022)
6. Selander, G., Mattsson, J., Palombini, F., Seitz, L.: Object Security for Constrained RESTful Environments (OSCORE). RFC 8613, IETF (2019)
7. Dworkin, M.: Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC. NIST SP 800-38D, NIST (2007). https://doi.org/10.6028/NIST.SP.800-38D
8. Kissel, R., et al.: Guidelines for Media Sanitization. NIST SP 800-88 Rev. 1, NIST (2014)
9. National Institute of Standards and Technology: Module-Lattice-Based Key-Encapsulation Mechanism Standard. FIPS 203, NIST (2024). https://doi.org/10.6028/NIST.FIPS.203

10. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. J. ACM **56**(6), Article 34 (2009). https://doi.org/10.1145/1568318.1568324
11. Lyubashevsky, V., Peikert, C., Regev, O.: On ideal lattices and learning with errors over rings. J. ACM **60**(6) (2013). https://doi.org/10.1145/2535925
12. Peikert, C.: Public-key cryptosystems from the worst-case shortest vector problem. In: Proc. 41st ACM STOC, pp. 333–342 (2009)
13. Gallager, R.G.: Low-density parity-check codes. IRE Trans. Inf. Theory **8**(1), 21–28 (1962). https://doi.org/10.1109/TIT.1962.1057683
14. Baldi, M., et al.: Enhanced public key security for the McEliece cryptosystem. J. Cryptology **29**, 1–27 (2016)
15. Li, X., et al.: Lattice-based privacy-preserving and anonymous authentication scheme. IEEE Internet Things J. (2020)
16. Granjal, J., Monteiro, E., Silva, J.S.: Security for the internet of things: a survey. IEEE Commun. Surveys Tuts. **17**(3), 1294–1312 (2015)
17. Wang, H., et al.: Ring-LWE based two-factor authentication. IEEE Access (2019)
18. Lee, J., et al.: RLizard: Post-quantum key encapsulation. IEEE Access (2019)
19. Hu, J., et al.: QC-LDPC key encapsulation on FPGA. IEEE Trans. VLSI (2019)
20. Bormann, C., Ersue, M., Keranen, A.: Terminology for Constrained-Node Networks. RFC 7228, IETF (2014)
21. Reza, R., et al.: Lightweight IoT authentication with ECC session keys. IEEE Internet Things J. (2021)
22. Shim, K.A., et al.: Lightweight lattice-based authentication for IoT. IEEE Trans. Inf. Forensics Secur. (2019)
23. Mundhe, P., et al.: Lightweight post-quantum IoT authentication. IEEE IOTJ (2021)
24. Han, W., et al.: Efficient Ring-LWE key exchange for IoT. IEEE Access (2020)
25. Bellare, M., Rogaway, P.: Entity authentication and key distribution. In: Advances in Cryptology – CRYPTO 1993, LNCS, vol. 773, pp. 232–249. Springer, Heidelberg (1993)

session-amortization-matlab/simulation/results/sim_latency.pdf

Fig. 2. Cumulative computational cost — Kumari et al. (7.37 ms/packet) vs. SAKE-IoT (22.55 ms epoch + 0.068 ms/packet). Break-even at $N = 4$; at $N = 100$: 708.1 ms saved.

session-amortization-matlab/simulation/results/sim_bandwidth_bar.pdf

Fig. 3. Per-packet overhead: Kumari et al. $CT_0 = 408$ bits (QC-LDPC syndrome) vs. SAKE-IoT 224 bits (96-bit nonce + 128-bit TAG). Epoch-level auth overhead is identical in both schemes.

session-amortization-matlab/simulation/results/sim_energy.pdf

Fig. 4. Clock cycles per packet extending Kumari et al. Fig. 7. SAKE-IoT AEAD (74K cycles) vs. QC-LDPC HE (2.45M cycles): 33.1× reduction. HKDF: ≈6K; AES-256-GCM: ≈68K.